Árboles

Organizar datos jerárquicamente para que buscar sea barato

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Agenda de la clase

1. **El árbol general**: jerarquía, sin más
2. **El árbol binario** y sus cuatro recorridos
3. **El árbol binario de búsqueda (BST)**
   Orden total: buscar en O(log n)… si no degenera
4. **El montículo binario** (la Cola de Prioridad)
   O(log n) garantizado por la forma
5. **El AVL**: reparar el BST con rotaciones
6. **El B-tree**: cuando cada nivel es un acceso a disco
7. **Los adaptadores de la STL**: priority_queue, set y map

De dónde venimos

Del tema 2 damos por sabido:

La lista enlazada: un nodo, un valor y un puntero al **siguiente**
Coste amortizado y crecimiento geométrico del array dinámico

Del tema 3:

La **Pila**: vuelve aquí para convertir una recursión en un bucle
La **Cola**: vuelve aquí para recorrer un árbol por niveles
Los adaptadores de la STL (stack, queue, deque)

Un árbol es la lista del tema 2 con "hijos" en vez de "siguiente".

El hilo conductor del tema

Una única pregunta, respondida seis veces: **¿cómo organizo datos para que buscar sea barato?**

Bloque	Lo que aporta	El problema que deja abierto
Árbol general	Jerarquía, sin más	Aridad libre: incómodo de programar
Árbol binario	Aridad 2 y los recorridos	Ningún orden: buscar sigue siendo O(n)
BST	Orden total: buscar en O(log n)…	…si no degenera: ordenado → O(n)
Montículo	O(log n) por la forma	Renuncia a buscar por valor
AVL	Repara el BST con rotaciones	Binario: con millones en disco, la altura pesa
B-tree	Muchas claves por nodo	(cierra el tema: índices de bases de datos)

El árbol general

Jerarquía, sin más

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

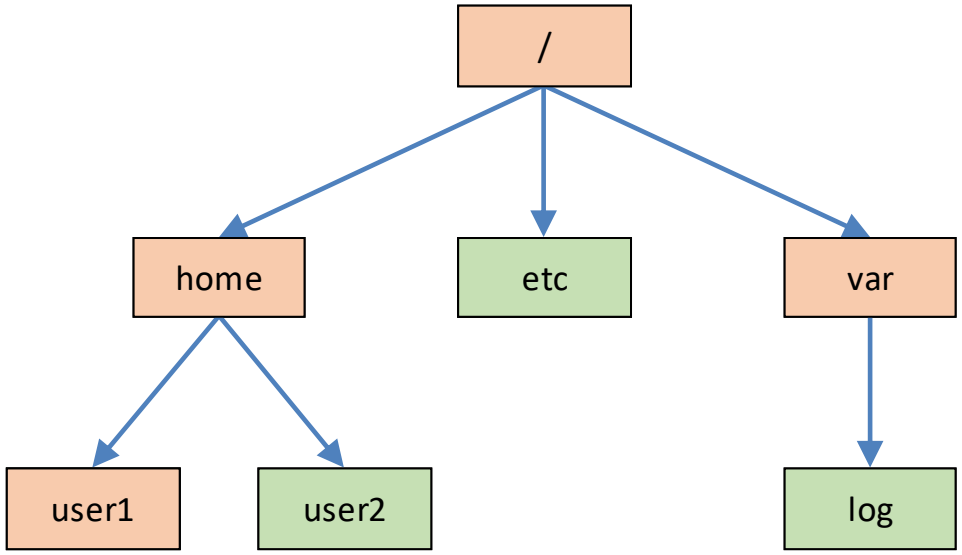
Vocabulario

Cada nodo tiene **cualquier número de hijos**.
La **raíz** no tiene padre; los demás,
exactamente uno.

Hoja: nodo sin hijos.
Hermanos: hijos del mismo padre.

Profundidad de un nodo: aristas desde la raíz.
Altura del árbol: la mayor profundidad.

Es la lista del tema 2 generalizada:
en vez de "el siguiente", "los hijos".



raíz: '/', el único sin padre

hijos de '/': home, etc, var (hermanos entre sí)

hojas: user1, user2, etc, log

profundidad(user1) = 2 -> altura del árbol = 2

nodos = 7

El TAD Árbol

Como con la Lista del tema 2: primero el **qué** (el contrato), después el **cómo**.

Un **Árbol** es una colección de nodos con una relación **jerárquica**:
- hay un nodo distinguido, la **raíz**, que no tiene padre
- todo nodo que no es la raíz tiene **exactamente un padre**
- los hijos de un nodo están **ordenados**: primero, segundo, …

¿Puede estar vacío? Por definición, un árbol tiene **uno o más** nodos.
Lo que sí puede estar vacío es el **contenedor**: un objeto Arbol recién creado aún no tiene raíz (raiz_ == nullptr), como primero_ == nullptr en la Lista.

Convenio: altura(vacío) = −1, para que una hoja salga con 1 + max(−1) = 0.

01_arbol_general.cpp

El TAD Árbol: operaciones

Operación	Pre	Post
vacio()	—	true si el árbol no tiene nodos
raiz()	!vacio()	la posición de la raíz
padre(p)	p válida	su padre, o **nula** si p es la raíz
primerHijo(p)	p válida	su primer hijo, o **nula** si es hoja
hermanoDerecho(p)	p válida	el siguiente hijo de su padre, o **nula**
etiqueta(p)	p válida	el valor guardado en p
crearRaiz(v)	vacio()	el árbol pasa a tener un nodo, con valor v
insertarHijo(p, v)	p válida	p tiene un hijo más, v, en la **última** posición

Se trabaja con **posiciones** (como la Posicion del iterador de listas):
la posición **nula** significa "no hay tal nodo".
Nada de esto habla de vectores ni de punteros: en el .cpp el contrato es la clase abstracta **Arbol**, con métodos virtual … = 0, como ListaDeEnteros.

01_arbol_general.cpp

Recorrer un árbol general en profundidad

Dos órdenes, según cuándo se visita la raíz respecto a sus subárboles:

Preorden: la raíz, y después cada subárbol de izquierda a derecha.
Inorden: no está definido aquí (¿la raíz va tras qué hijo?). Llega con el binario.
Postorden: todos los subárboles, y la raíz al final.

Cada uno tiene su uso natural en un sistema de ficheros:

Preorden = listar el árbol (tree, ls -R): el directorio antes que su contenido
Postorden = calcular el tamaño (du): un directorio pesa lo que suman sus hijos, así que hay que haberlos visitado antes

Preorden del ejemplo: / home user1 user2 etc var log

¿Posibles implementaciones?



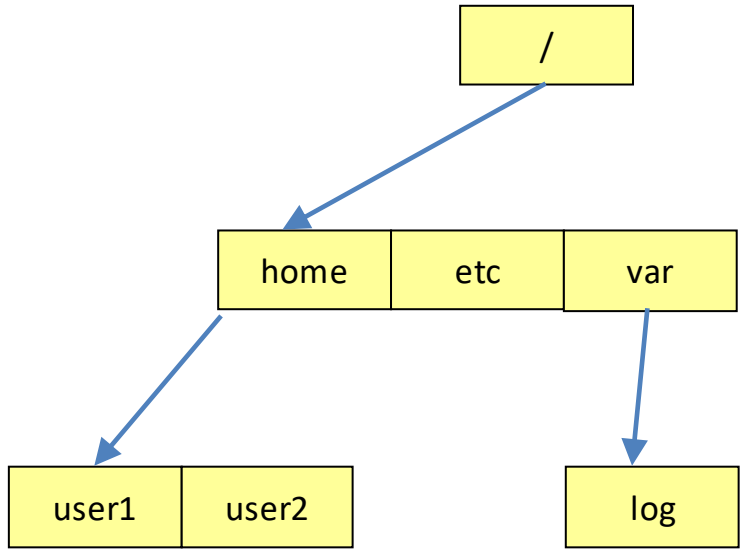
Dos representaciones

Árbol general

```
// A: lista de hijos
struct NodoLista {
    std::string etiqueta;
    std::vector<NodoLista*> hijos;
    NodoLista* padre = nullptr;
};
```

Implementa el TAD (clase ArbolLista):
- insertarHijo: O(1) amortizado
- primerHijo: O(1)
- hermanoDerecho: O(k)
- padre: O(1)

NodoLista



Cada nodo guarda un vector con TODOS sus hijos (aridad variable)

Dos representaciones

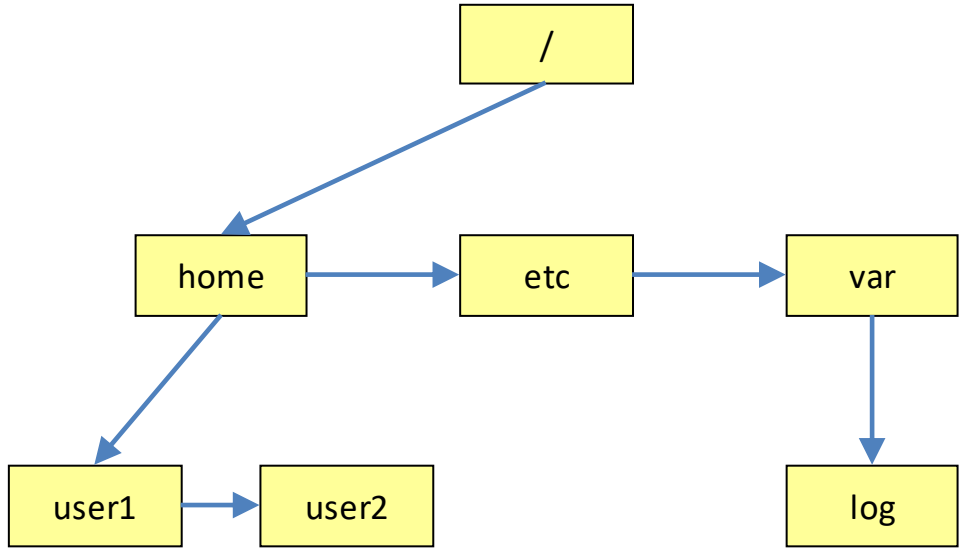
Árbol general

```
// B: primer hijo / hermano derecho
struct NodoPHHD {
    std::string etiqueta;
    NodoPHHD* primerHijo_     = nullptr;
    NodoPHHD* hermanoDerecho_ = nullptr;
};
```

Implementa el TAD (clase ArbolPHHD):
- insertarHijo: O(k)
- primerHijo: O(1)
- hermanoDerecho: O(1)
- padre: O(n)

O(k) es lineal en el número de hijos; como k puede llegar a n−1, en el peor caso es O(n). Solo sería constante si se limita la aridad

NodoPHHD



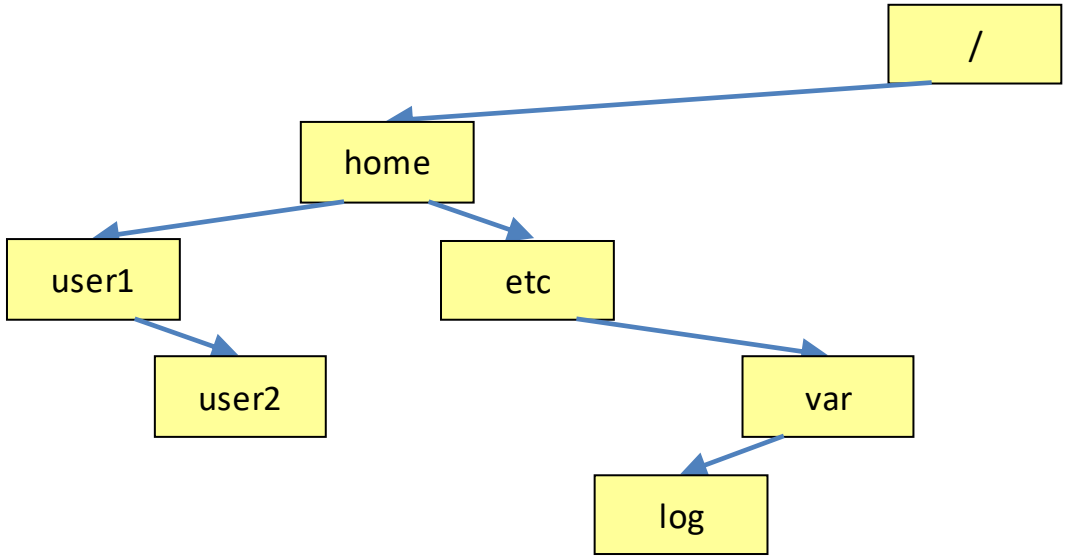
primerHijo_ baja un nivel; hermanoDerecho_ se queda en el mismo
B: solo el PRIMER hijo (abajo) y el SIGUIENTE hermano (a la derecha)

Observación: Nuestro árbol n-ario como binario

Árbol general

```
// B: primer hijo / hermano derecho
struct NodoPHHD {
    std::string etiqueta;
    NodoPHHD* primerHijo_     = nullptr;
    NodoPHHD* hermanoDerecho_ = nullptr;
};
```

NodoPHHD



girado 45°: izq = primerHijo_, der = hermanoDerecho_

¡es un ÁRBOL BINARIO! Cualquier árbol n-ario cabe en uno binario

Ejercicio 1: el mismo árbol, dos representaciones

Con la representación primer-hijo / hermano-derecho, preorden baja a primerHijo y sigue a hermanoDerecho.

¿En qué dirección aumenta la profundidad: hacia el hijo o hacia el hermano?
¿Por qué recorrer así sigue dando un preorden correcto del árbol general?

primerHijo_ baja de nivel: es un hijo de verdad (profundidad + 1).
hermanoDerecho_ se queda en el mismo nivel: es otro hijo del mismo padre,
no un descendiente (misma profundidad).

Lo que hay que llevarse del bloque:
cualquier árbol n-ario se puede representar como un árbol binario sin perder nada.
Por eso, a partir de aquí, todo se explica sobre árboles binarios.

01_arbol_general.cpp

El precio en memoria de cada representación

Representación	Cada nodo guarda	Coste por nodo
Lista de hijos	vector<Nodo*> de hijos y padre	Variable: el vector tiene su bloque
Primer-hijo / hermano	dos punteros	**Fijo**, tenga 0 o 100 hijos

Teniendo en cuenta que sizeof(std::string) vale 24:
sizeof(NodoLista) = 56 bytes + el bloque del vector en el heap (capacidad, no tamaño). ¿De dónde sale ese tamaño*?
sizeof(NodoPHHD) = 40 bytes siempre: dos punteros (8*2=16), cero bloques adicionales

Las dos dan el mismo recorrido, la misma altura (2) y los mismos nodos (7).
Lo que cambia es **cuántas reservas de memoria** hace falta para guardarlo.

01_arbol_general.cpp

*vector tiene tres punteros internos: begin, end, capacity (nosotros usábamos un entero). Cada NodoLista tiene su propio vector, por lo que cada nodo tiene un bloque de memoria adicional para su vector de hijos.

Mismo contrato, distinto coste

Operación	A (vector de hijos)	B (primer hijo / hermano)
primerHijo(p)	O(1)	O(1)
hermanoDerecho(p)	O(k): buscar p en el vector del padre	**O(1)**
padre(p)	**O(1)**: el nodo guarda su padre	O(n): buscarlo desde la raíz
insertarHijo(p, v)	**O(1)** amortizado (push_back)	O(k): recorrer los hermanos

k = hijos del padre; n = nodos del árbol.
A necesita saber quién es el padre para dar hermanoDerecho: por eso NodoLista guarda también un puntero **padre** (y pasa de 48 a 56 bytes).

¿Qué añadirías a NodoPHHD para que padre(p) fuera O(1)? ¿Qué pierde B?

Un tercer puntero **padre_**: 8 bytes más por nodo, e insertarHijo debe mantenerlo.
Es la decisión del tema 2 entre lista simple y doble: un puntero más por nodo, a cambio de poder ir hacia atrás.

01_arbol_general.cpp

El árbol binario

Aridad 2, y las cuatro formas de recorrerlo

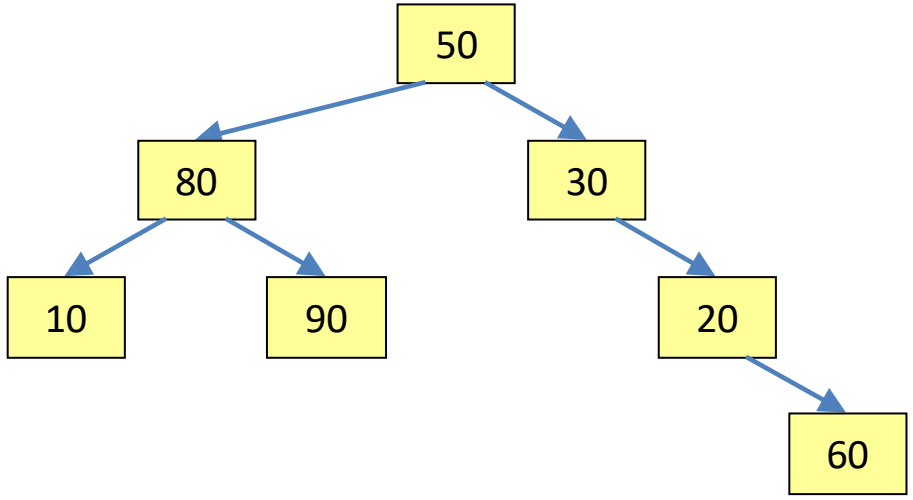
AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

El árbol binario

Cada nodo tiene **como mucho dos hijos**, y **no son intercambiables**: izquierdo y derecho son posiciones distintas.

```
struct Nodo {
    int valor;
    Nodo* izq = nullptr;
    Nodo* der = nullptr;
};
```



El malentendido más común del tema: un árbol binario cualquiera **no** tiene por qué estar ordenado. El de la derecha no lo está, a propósito: el orden llega en el bloque 3.

02_arbol_binario_recorridos.cpp

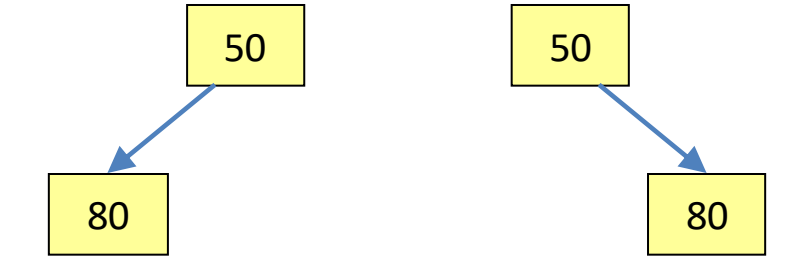
El TAD Árbol Binario

El TAD del bloque 1 con otra navegación:
primerHijo(p) / hermanoDerecho(p)
-> **izquierdo(p) / derecho(p)**

Y el **árbol vacío** pasa a ser obligatorio:
forma parte de la definición, que es recursiva.

Un árbol binario es, o bien el árbol **vacío**,
o bien una **raíz** con dos subárboles binarios
(izquierdo y derecho), cualquiera de ellos vacío.

Cada if (n == nullptr) del código es
el caso base de esa definición.



solo hijo izquierdo

solo hijo derecho

mismos nodos, árboles DISTINTOS:

sin el árbol vacío no se distinguirían

02_arbol_binario_recorridos.cpp

Cuatro recorridos, una sola recursión

Recorrido	Orden	Se programa con
Preorden	visitar, izquierdo, derecho	Recursión (o una pila explícita)
Inorden	izquierdo, visitar, derecho	Recursión (o una pila explícita)
Postorden	izquierdo, derecho, visitar	Recursión (o una pila explícita)
Por niveles	nivel a nivel	Una **cola** (tema 3)

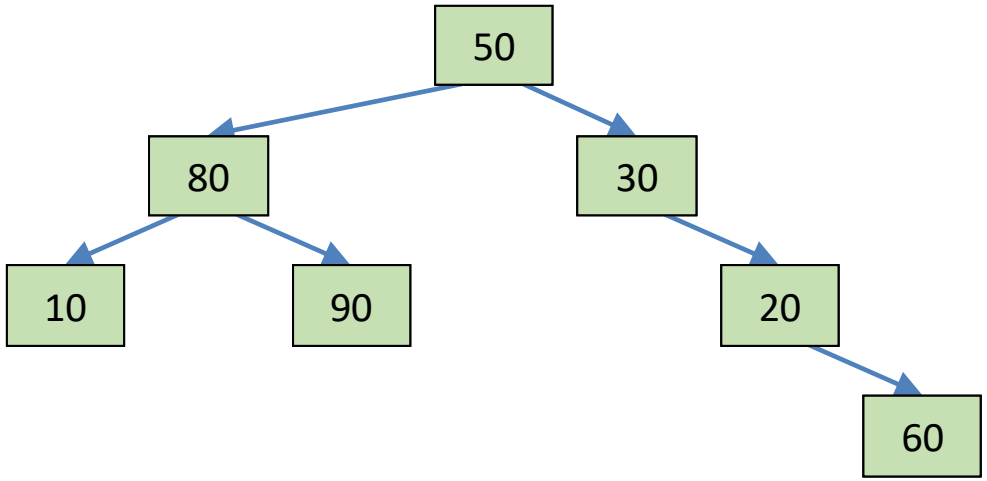
Los tres en profundidad son **la misma función**: solo cambia de sitio la línea visitar(n). Recorrer cuesta siempre **O(n)**: cada nodo se visita una vez.

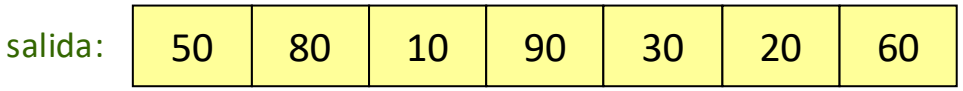
Recorridos

Árbol binario: recorridos

```
void preorden(Nodo* n, vector<int>& s) {
    if (n == nullptr) return;
    s.push_back(n->valor);   // visitar
    preorden(n->izq, s);
    preorden(n->der, s);
}
```

preorden()





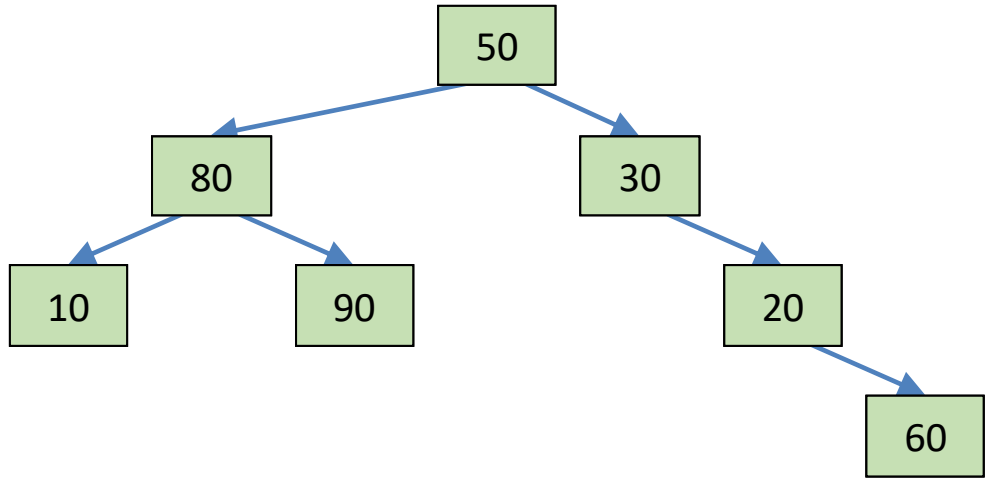
la raíz sale la PRIMERA: sirve para copiar o guardar un árbol

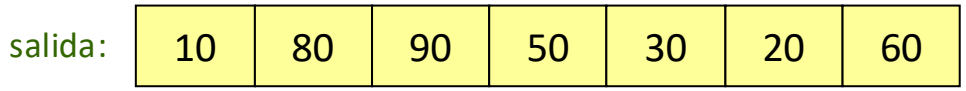
Recorridos

Árbol binario: recorridos

```
void inorden(Nodo* n, vector<int>& s) {
    if (n == nullptr) return;
    inorden(n->izq, s);
    s.push_back(n->valor);   // visitar
    inorden(n->der, s);
}
```

inorden()





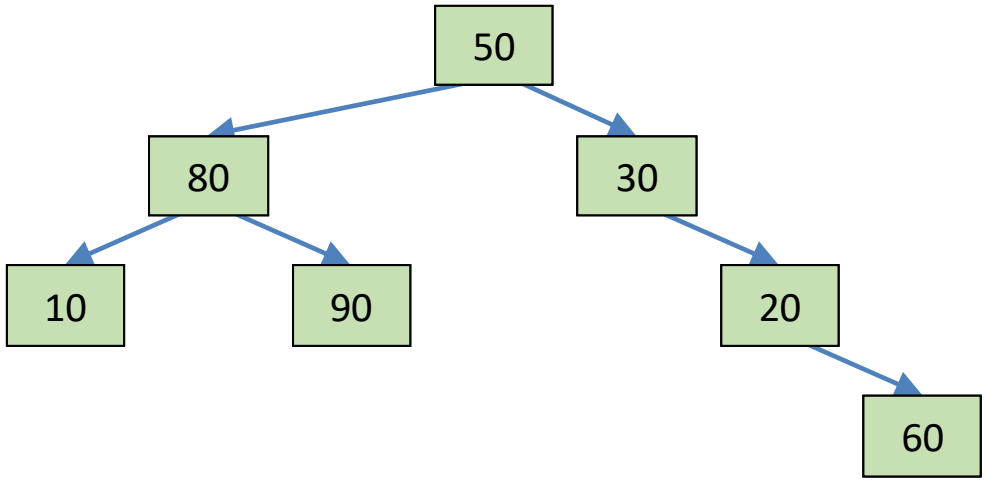
10 80 90 50 30 20 60: NO sale ordenado. Un árbol binario no tiene orden

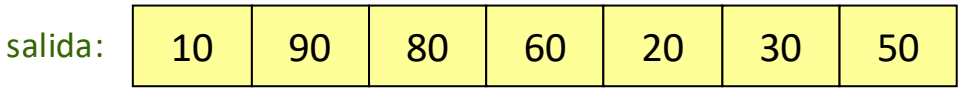
Recorridos

Árbol binario: recorridos

```
void postorden(Nodo* n, vector<int>& s) {
    if (n == nullptr) return;
    postorden(n->izq, s);
    postorden(n->der, s);
    s.push_back(n->valor);   // visitar
}
```

postorden()





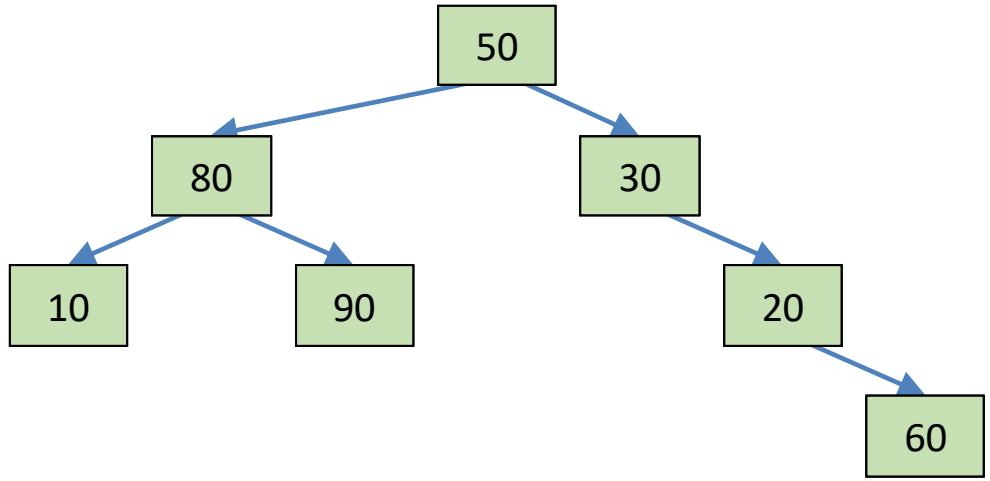
la raíz sale la ÚLTIMA: sirve para liberar memoria (los hijos antes)

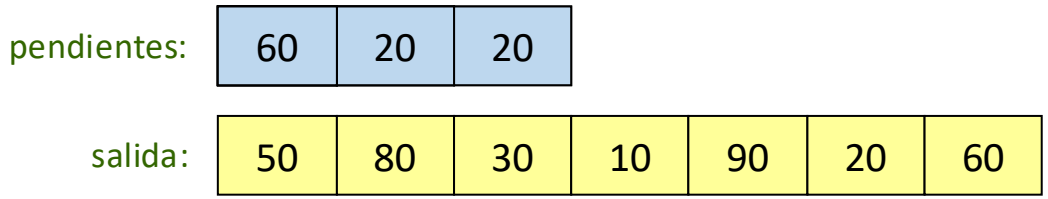
Recorridos

Árbol binario: recorridos

```
vector<int> porNiveles(Nodo* raiz) {
   vector<int> salida;
   if (raiz == nullptr) return salida;
   queue<Nodo*> pendientes;
   pendientes.push(raiz);
   while (!pendientes.empty()) {
      Nodo* actual = pendientes.front();
      pendientes.pop();
      salida.push_back(actual->valor);
      if (actual->izq) pendientes.push(actual->izq);
      if (actual->der) pendientes.push(actual->der);
   }
   return salida;
}
```

porNiveles()





Ejercicio 2: el inorden sin recursión

En el tema 3 lo dijimos sin demostrarlo con código:
"toda recursión se puede convertir en un bucle con una pila explícita",
porque una llamada recursiva no es más que la pila de llamadas del programa.

Tu tarea: completar inordenIterativo() con un std::stack<Nodo*>,
sin ninguna llamada recursiva.

```
std::vector<int> inordenIterativo(Nodo* raiz) {
    std::vector<int> salida;
    // TODO: implementar con std::stack<Nodo*>
    return salida;
}
```

Antes de escribir código: el inorden recursivo baja por la izquierda antes de visitar nada. ¿Qué papel juega la pila en esa bajada? ¿Cuándo se "visita" un nodo?

02_arbol_binario_recorridos.cpp

El árbol binario de búsqueda

Orden total: la búsqueda binaria, enlazada

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

La propiedad BST

Una única propiedad añadida al árbol binario. Para **todo** nodo:

todo su subárbol izquierdo < nodo < **todo** su subárbol derecho

Es la búsqueda binaria llevada a una estructura enlazada: en cada nodo se descarta la mitad del árbol **sin mirarla**, aunque los datos no estén contiguos en memoria.

Operación	**Coste (árbol equilibrado)**
buscar(v)	O(log n)
insertar(v)	O(log n)
borrar(v)	O(log n)
inorden()	O(n), y sale **ordenado**: es la propiedad BST leída al revés

03_arbol_binario_busqueda.cpp

Cambia el TAD, no solo la estructura

En los bloques 1 y 2, quien usaba el árbol **navegaba** por él:
raiz(), izquierdo(p), derecho(p), padre(p)…

En el BST, no. Quien lo usa pide cosas **por valor**, y no ve ni un nodo:
insertar(v), buscar(v), borrar(v)

El TAD que se ofrece es un **Conjunto** (o un **Diccionario**, si cada clave lleva un valor asociado); el árbol es solo su **implementación**.

Por eso ArbolBusqueda esconde Nodo y raiz_ en su parte privada: las posiciones ya no forman parte del contrato. Es lo que hace **std::set**.

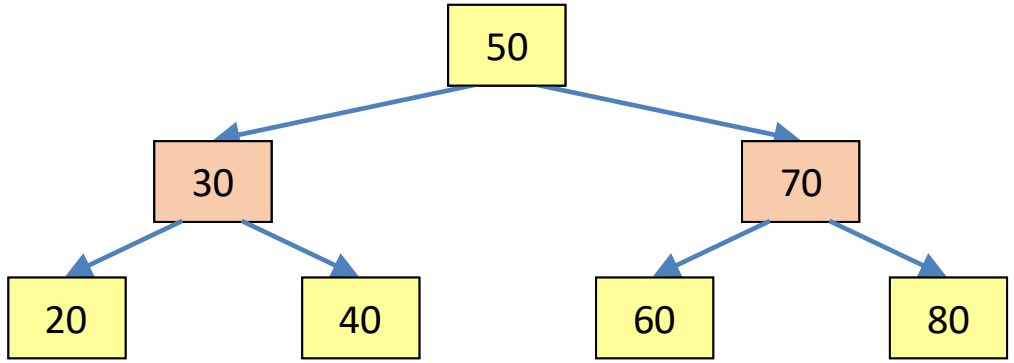
03_arbol_binario_busqueda.cpp

Implementación

Árbol binario de búsqueda

```
Nodo* insertarEn(Nodo* n, int v) {
    if (n == nullptr)
        return new Nodo(v);
    if (v < n->valor)
        n->izq = insertarEn(n->izq, v);
    else if (v > n->valor)
        n->der = insertarEn(n->der, v);
    return n;      // repetido: no se inserta
}
```

insertar(v)



insertados 7 nodos

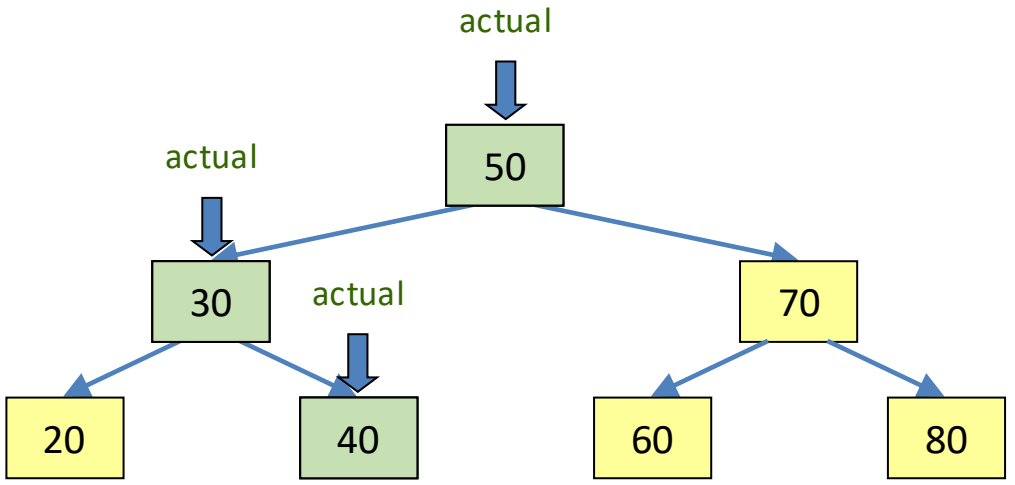
Implementación

Árbol binario de búsqueda

```
bool buscar(int v) const {
   Nodo* actual = raiz_;
   while (actual != nullptr) {
      if (v == actual->valor) return true;
      actual = (v < actual->valor)
            ? actual->izq : actual->der;
   }
   return false;
}
```

Cada comparación descarta un subárbol entero sin mirarlo: O(altura).

buscar(v)



buscar(45): accede a [illegible] nodos de 7 nodos

Implementación

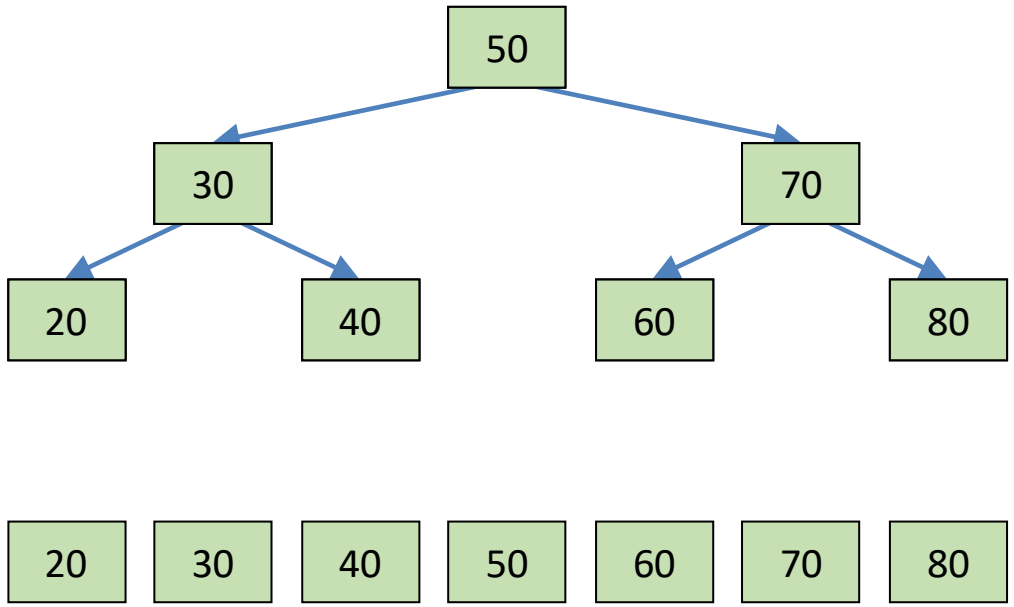
Árbol binario de búsqueda

```
void inorden(Nodo* n, vector<int>& s) {
    if (n == nullptr) return;
    inorden(n->izq, s);   // todo lo MENOR
    s.push_back(n->valor);
    inorden(n->der, s);   // todo lo MAYOR
}
```

O(n): cada nodo se visita una vez.

Y sale ordenado: al insertar, la propiedad BST decide DÓNDE va cada valor; al recorrer en inorden, esa misma desigualdad se lee como el ORDEN de salida.

inorden()



cada nodo cae en su columna: la proyección del BST ES la secuencia ordenada

Ejercicio 3: borrar, el caso que hay que pensar dos veces

Completar borrarEn(). Los tres casos, de menos a más difícil:

1. **Nodo hoja**: se quita sin más.

2. **Nodo con un solo hijo**: el nodo se "salta": ese hijo ocupa su lugar.

3. **Nodo con dos hijos**: no se puede quitar sin más (¿qué hueco dejaría?).
   Busca el **sucesor inorden:** el mínimo del subárbol derecho: el más pequeño de los que son mayores que el nodo (también nos vale el caso simétrico: el más grande de los que son menores que el nodo), copia su valor a este nodo, y borra recursivamente ese sucesor de donde estaba.

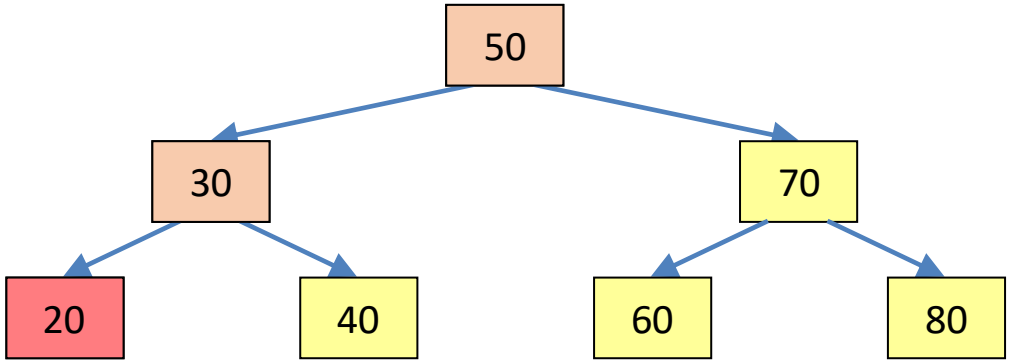
La pregunta que debes poder responder: ¿por qué el borrado recursivo del sucesor del caso 3 nunca vuelve a caer en el propio caso 3?

03_arbol_binario_busqueda.cpp

Implementación

BST: caso 1, un nodo hoja

borrar(20)



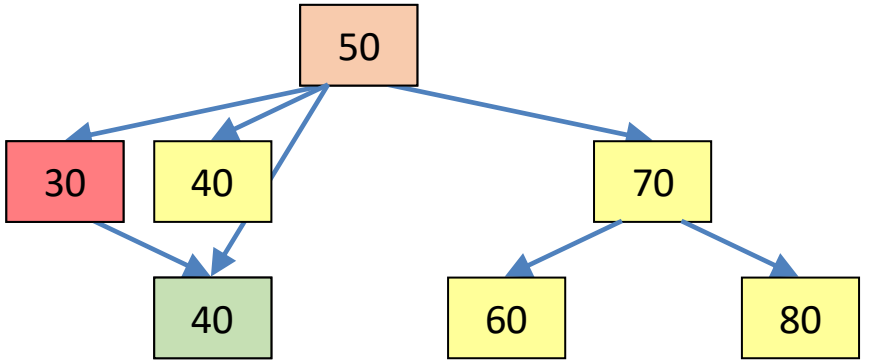
caso 1: 20 < 50, 20 < 30, 20 encontrado. No tiene hijos

```
if (n->izq == nullptr && n->der == nullptr) {
    delete n;
    n = nullptr;
}
```

Implementación

BST: caso 2, un solo hijo

borrar(30)

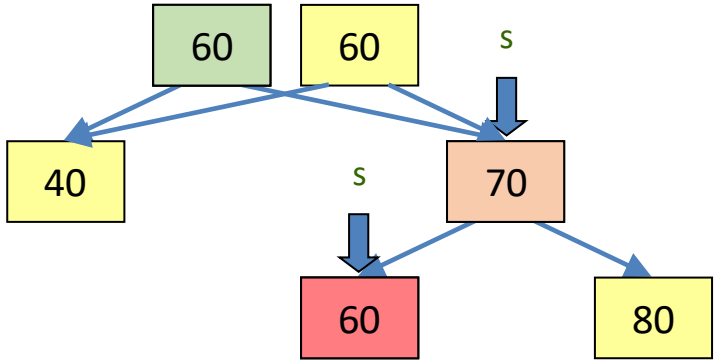


40 sube a su nivel, pasa a ser hijo izquierdo de 50 (antes era hijo izquierdo de 30), igual que antes

Implementación

BST: caso 3, dos hijos

borrar(50)



¿Por qué el caso 3 nunca se repite?

El borrado recursivo del sucesor, ¿puede volver a caer en el caso 3?

No. El mínimo de un subárbol se encuentra bajando siempre por la izquierda.

Por construcción, ese nodo **nunca tiene hijo izquierdo**:
si lo tuviera, ese hijo sería aún más pequeño, y el mínimo sería él.

Así que al borrarlo cae siempre en el **caso 1** (sin hijo derecho)
o en el **caso 2** (solo con el derecho).

Nunca hace falta un cuarto caso, ni una segunda vuelta del tercero.

Implementación

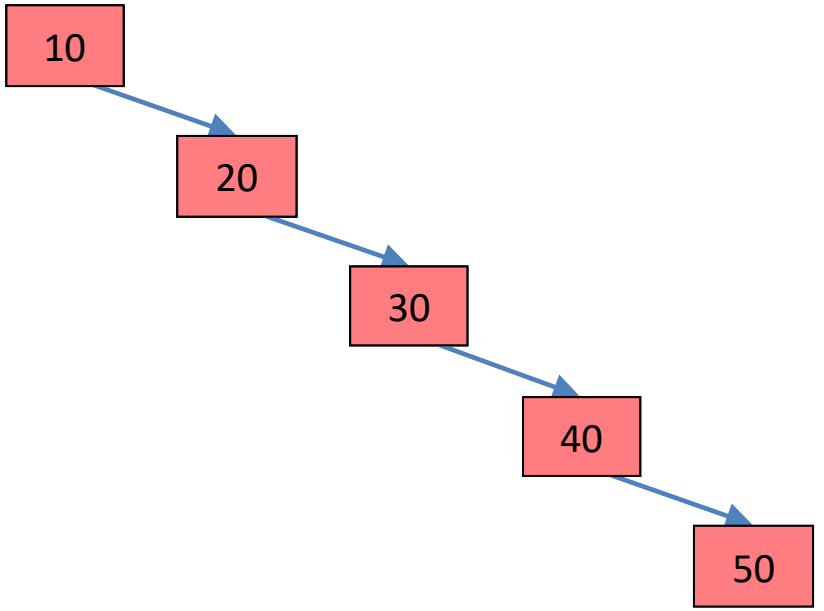
BST: el caso patológico

```
for (int x : {10, 20, 30, 40, 50})
    arbol.insertar(x);
```

Cada nodo nuevo cuelga siempre del mismo lado: altura = n - 1.

Es una lista enlazada disfrazada de árbol: buscar vuelve a ser O(n).

insertar en orden



insertar(40): mayor que todos → siempre a la derecha. Altura 4

El aviso: orden de inserción frente a altura

n	altura (aleatorio)	altura (YA ordenado)
100	11	99
1000	21	999
10000	30	9999
100000	39	99999

Con datos al azar, la altura crece como **log n**.
Con datos ya ordenados, es **exactamente n – 1**.

Los dos bloques siguientes resuelven este problema, cada uno a su manera:
el **montículo** cambia de estructura; el **AVL** repara este mismo árbol.

03_arbol_binario_busqueda.cpp

Dos maneras de garantizar O(log n)

El BST deja abierta la pregunta: **¿cómo garantizo O(log n)?** Hay dos respuestas:

Cambiar de estructura (montículo):
exige una forma fija —árbol completo— que da la altura gratis,
y a cambio **renuncia** a algo que el BST tenía: buscar por valor.

Reparar la que ya tienes (AVL):
mantiene el orden total del BST y paga trabajo extra en cada inserción
(rotaciones) para que nunca degenere.

Las dos son legítimas. Saber cuál elegir es una de las lecciones del tema.

El montículo binario

La Cola de Prioridad: O(log n) por la forma

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

El TAD Cola de Prioridad

El orden de salida no lo decide la llegada: lo decide la **prioridad**.
Sale siempre el más prioritario.

Operación	**Precondición**	**Postcondición**
insertar(v)	—	v está en la colección
maximo()	!vacia()	Devuelve el mayor; no modifica
extraerMaximo()	—	Si no estaba vacía, elimina **una** aparición del mayor

Implementación	**insertar**	**extraerMaximo**
Lista sin ordenar	O(1)	**O(n)**
Lista ordenada	**O(n)**	O(1)
Montículo	O(log n)	O(log n)

La estructura: forma antes que orden

Un **montículo binario** es un árbol binario con dos propiedades:

1. **Completo**: todos los niveles llenos menos el último, que se rellena de izquierda a derecha. La altura es exactamente $\lfloor \log_2 n \rfloor$: no es una esperanza estadística como en el BST, **es una consecuencia de la forma**.

2. **Propiedad de montículo**: todo padre es ≥ que sus hijos.
   No es un árbol de búsqueda: entre hermanos no hay ningún orden.

Al ser completo, no necesita punteros: **cabe en un array**.

padre(i) = (i − 1) / 2 izquierdo(i) = 2i + 1 derecho(i) = 2i + 2

04_cola_prioridad_heap.cpp

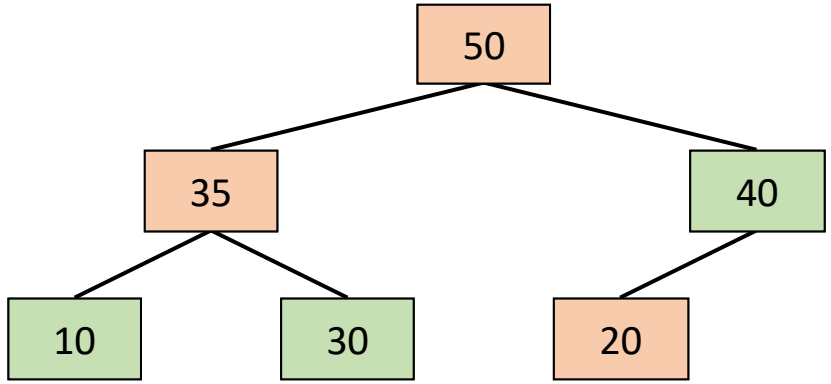
Implementación

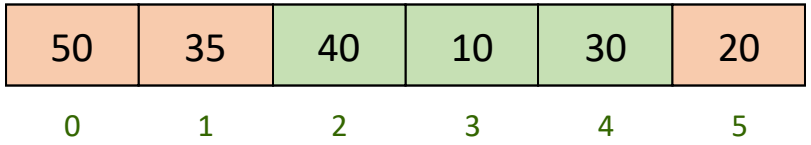
Montículo binario

```
int padre(int i)     { return (i - 1) / 2; }
int izquierdo(int i) { return 2 * i + 1; }
int derecho(int i)   { return 2 * i + 2; }
```

No hay ni un puntero:
- la relación padre-hijo es aritmética.
- Por eso las aristas no llevan punta.

el array ES un árbol





invariante: todo padre >= sus hijos -> datos_[0] es el máximo

padre(5) = (5 - 1) / 2 = 2: el padre de 20 es 40

izquierdo(1) = 3, derecho(1) = 4: los hijos de 35 son 10 y 30

nivel 0: 50 | nivel 1: 35 40 | nivel 2: 10 30 20

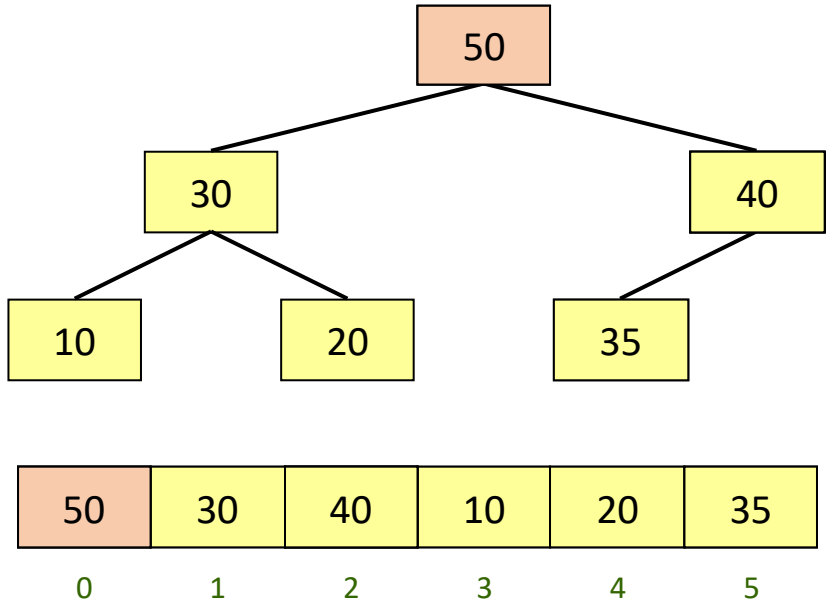
Implementación

Montículo binario

```
void insertar(int valor) {
    datos_.push_back(valor);
    subir(datos_.size() - 1);
}
void subir(int i) {
    while (i > 0) {
        if (datos_[padre(i)] >= datos_[i])
            break;
        swap(datos_[padre(i)], datos_[i]);
        i = padre(i);
    }
}
```

O(log n): sube como mucho la altura.

insertar(50)



invariante: todo padre >= sus hijos -> datos_[0] es el máximo

i = 0 para 2(50) [illegible] el log n

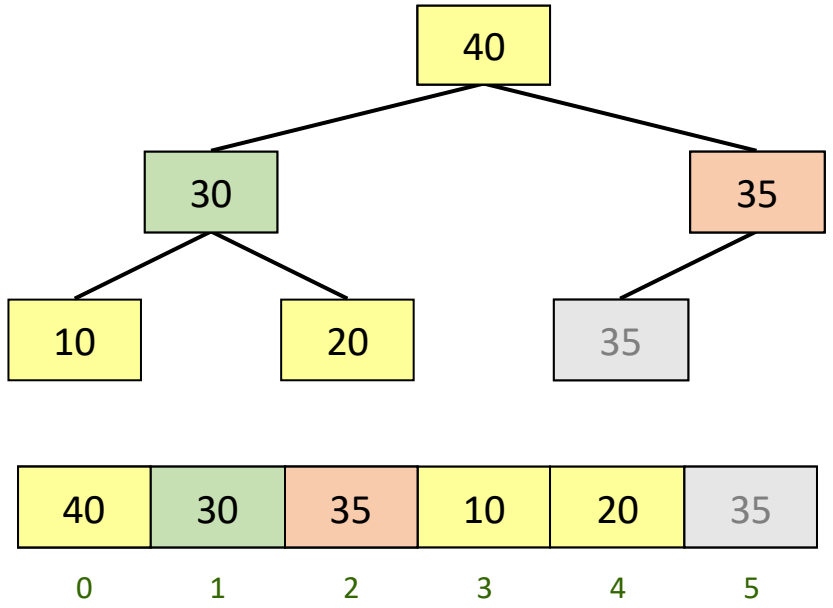
Implementación

Montículo binario

```
bool extraerMaximo() {
    if (datos_.empty()) return false;
    datos_[0] = datos_.back();
    datos_.pop_back();
    if (!datos_.empty()) bajar(0);
    return true;
}
void bajar(int i) {
    // comparar con iz y de;
    // intercambiar con el MAYOR de los
    // dos mientras alguno sea mayor
}
```

O(log n): baja como mucho la altura.

extraerMaximo()



invariante: todo padre >= sus hijos -> datos_[0] es el máximo

pop_back() baja(0) hijos MAYOR padre >= sus hijos

Lo que un montículo no sabe hacer

A cambio de garantizar O(log n) por la forma, renuncia a cosas que el BST sí hacía:

Buscar un valor cualquiera: O(n). No es un árbol de búsqueda;
entre hermanos no hay orden y no se puede descartar ninguna rama.

Recorrer en orden: no hay inorden útil. Hay que ir vaciándolo.

Cambiar la prioridad de un elemento concreto sin saber dónde está.

Solo sirve para **el extremo**. Pero eso, lo hace muy bien.

04_cola_prioridad_heap.cpp

Tres problemas interesantes con montículos

Creación a partir de un vector de elementos:

- Insertando uno a uno del vector al montículo.
- Volcando todos los elementos al montículo y reorganizando (Floyd 1964)

Ordenación basada en montículo (Heapsort)

- Cálculo de complejidad en tiempo y espacio.

Top k: Qué hacer cuando queremos mantener los mejores elementos de un vector enorme

- Por ejemplo, de los últimos 5 millones de ventas, ¿cuáles han sido las 10 mejores?

04_cola_prioridad_heap.cpp

1) Construir un montículo: dos maneras

Partimos de un **array desordenado** con n elementos y queremos convertirlo en montículo.

	Insertar uno a uno	**Floyd (1964)**
Idea	push_back + subir, n veces	bajar desde el último padre hasta la raíz
Cada nodo se mueve	lo que mide su PROFUNDIDAD (sube)	lo que mide su ALTURA (baja)
Coste	**O(n log n) en el peor caso**	**O(n) siempre**

Las dos observaciones de Floyd:

1. Un array, leído con 2i+1 y 2i+2, **ya es un árbol completo**: solo falla el orden.
2. Las hojas ya son montículos de un elemento. En un árbol completo son **la mitad del array** (índices n/2 … n−1): la mitad del trabajo está hecha.

05_heapify_heapsort.cpp

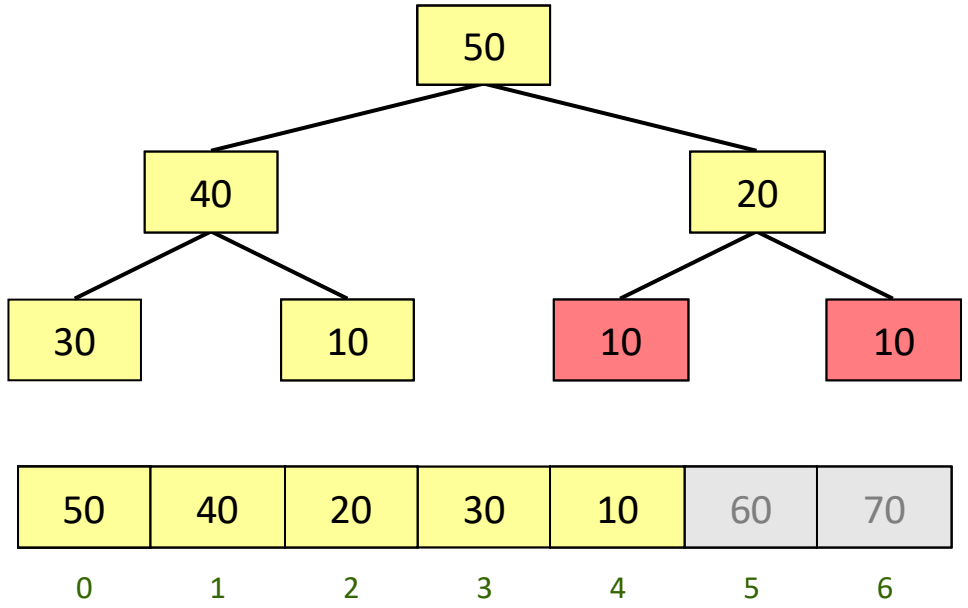
2) Heapsort: Ordenación por montículo

Montículo: heapsort

```
void heapsort(vector<int>& v) {
   construir(v);            // O(n)
   for (int fin = v.size() - 1; fin > 0; fin--) {
      swap(v[0], v[fin]);   // el máximo, a su sitio
      bajar(v, 0, fin);     // O(log n)
   }
}
```

O(n log n) siempre, sin memoria extra.
Pierde contra std::sort (x1.4/1.7) por localidad de caché: salta por el array.

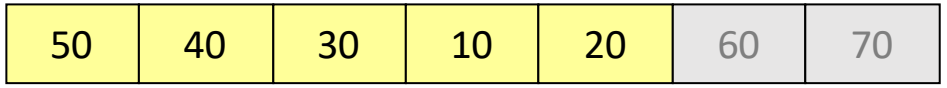
heapsort



la raíz (70) es el mayor (10)

2) Heapsort: dos preguntas

¿Por qué un montículo de MÁXIMOS para ordenar de menor a mayor?



← montículo | ya ordenado (gris) →

Cada paso saca el máximo de la raíz y lo deja en la última casilla del montículo, que es su sitio definitivo.
El array se parte en dos: el montículo encoge y la zona ordenada crece por la derecha.
Todo dentro del mismo array: memoria O(1).

¿Por qué std::sort no es un heapsort, si es O(n log n) siempre?

Por la **caché**: bajar salta de i a 2i+1, luego a 4i+3…, a zonas lejanas del array.
Quicksort recorre seguido. Medido: heapsort tarda entre **1,5 y 2 veces** más.

Pero no se tira: std::sort es un introsort, un quicksort que **cambia a heapsort** si va camino de su peor caso $O(n^2)$, justo porque heapsort lo garantiza. https://en.wikipedia.org/wiki/Introsort

05_heapify_heapsort.cpp

3) Top-k mejores

De los últimos 5 millones de ventas, ¿cuáles han sido las 10 mejores?

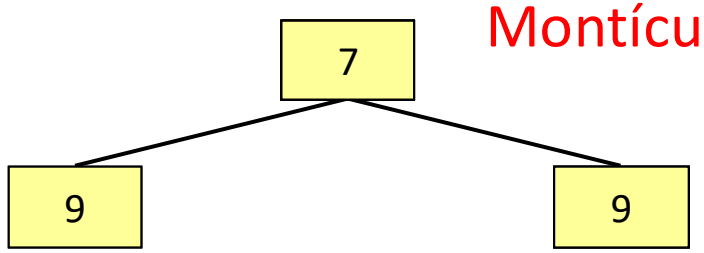
Montículo: top-k

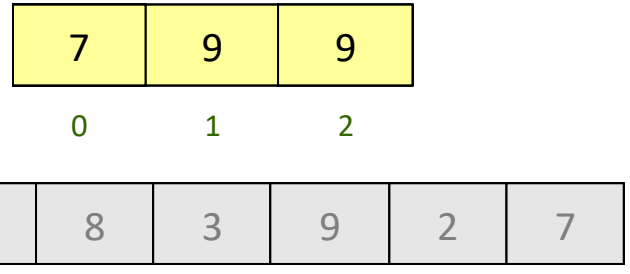
```
// los k MAYORES con un montículo
// de MÍNIMOS: top() es el listón
priority_queue<int, vector<int>, greater<int>> mejores;
for (int x : datos) {
    if (mejores.size() < k)
        mejores.push(x);
    else if (x > mejores.top()) {
        mejores.pop();
        mejores.push(x);
    }
}
```

O(n log k) y memoria O(k): el montículo nunca pasa de k elementos, y cada dato se mira una vez y se tira.

top-k, con k = 3

Montículo de mínimos





El árbol AVL

Reparar el BST en vez de cambiar de estructura

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

AVL: el invariante

Adelson-Velskii y Landis, 1962. Mantener el orden total del BST, y **reparar** el desequilibrio en el momento en que aparece, con una **rotación**.

El invariante, para todo nodo: $|\ \text{altura(der)} - \text{altura(izq)}\ | \leq 1$

Factor de equilibrio: fe(n) = altura(der) − altura(izq)
positivo: pesa la derecha; negativo: pesa la izquierda; ±2: hay que rotar

Cada nodo guarda su altura (un entero más) para calcularlo en O(1).

Resultado: la altura nunca supera **1,44 $\log_2$ n**, insertes en el orden que insertes.

06_arbol_avl.cpp

Cuatro casos, dos rotaciones

Caso	Diagnóstico	Arreglo
LL	Pesa la izquierda, y el problema está a la izquierda de esa izquierda	Una rotación simple a la derecha
RR	Simétrico	Una rotación simple a la izquierda
LR	Pesa la izquierda, pero el problema está a la **derecha** de esa izquierda	Rotar el hijo izquierdo a la izquierda (LR → LL), y luego LL
RL	Simétrico	Rotar el hijo derecho a la derecha (RL → RR), y luego RR

Las rotaciones se aplican **al volver de la recursión**: arreglan siempre el desequilibrio más bajo, el más cercano al nodo recién insertado.

Implementación

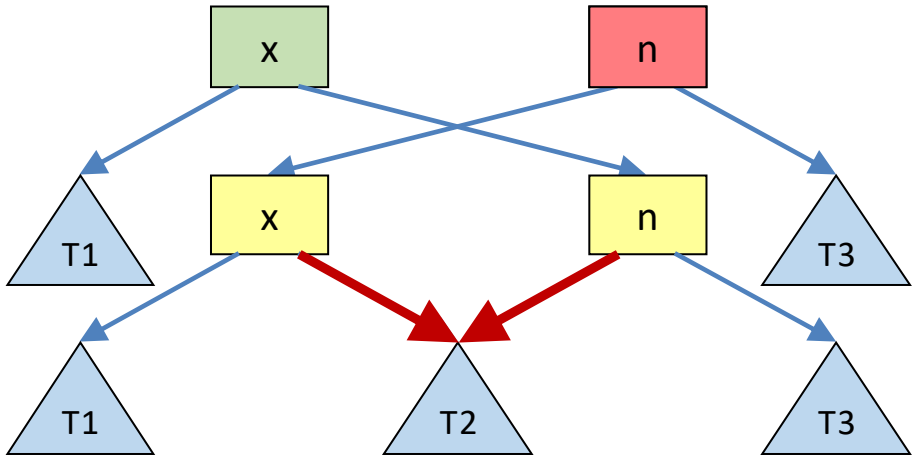
Árbol AVL

```
Nodo* rotarDerecha(Nodo* n) {
   Nodo* nuevaRaiz = n->izq;
   n->izq = nuevaRaiz->der;
   nuevaRaiz->der = n;
   actualizarAltura(n);   // n primero:
   actualizarAltura(nuevaRaiz);
   return nuevaRaiz;      // el más hondo
}
```

Tres punteros y dos alturas: O(1).

El inorden T1 x T2 n T3 no cambia: sigue siendo BST.

rotarDerecha(n)



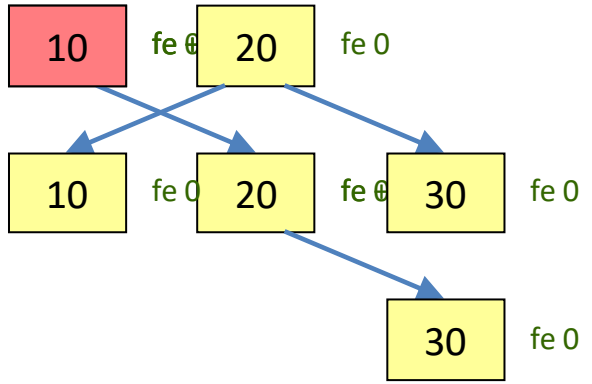
...así T2 pasa de colgar de la izquierda de x a colgar de la derecha de n, y n baja

Implementación

Árbol AVL

```
Nodo* reequilibrar(Nodo* n) {
   int fe = factorEquilibrio(n);
   if (fe < -1) {
      if (factorEquilibrio(n->izq) <= 0)
         return rotarDerecha(n);   // LL
      n->izq = rotarIzquierda(n->izq);
      return rotarDerecha(n);      // LR
   }
   if (fe > 1) {
      if (factorEquilibrio(n->der) >= 0)
         return rotarIzquierda(n); // RR
      n->der = rotarDerecha(n->der);
      return rotarIzquierda(n);    // RL
   }
   return n;
}
```

caso RR: 10 20 30



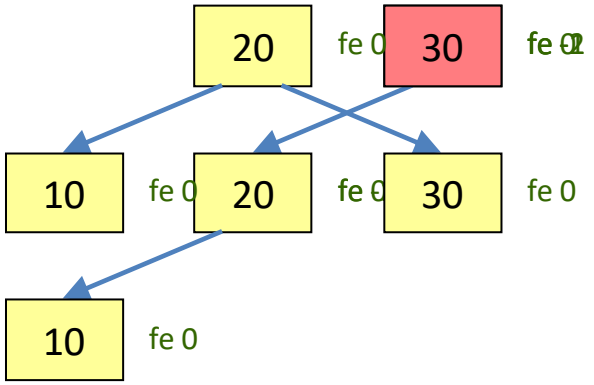
fe(10)=-2 y fe(20)=-1 caso RR

Implementación

Árbol AVL

```
Nodo* reequilibrar(Nodo* n) {
   int fe = factorEquilibrio(n);
   if (fe < -1) {
      if (factorEquilibrio(n->izq) <= 0)
         return rotarDerecha(n);   // LL
      n->izq = rotarIzquierda(n->izq);
      return rotarDerecha(n);      // LR
   }
   if (fe > 1) {
      if (factorEquilibrio(n->der) >= 0)
         return rotarIzquierda(n); // RR
      n->der = rotarDerecha(n->der);
      return rotarIzquierda(n);    // RL
   }
   return n;
}
```

caso LL: 30 20 10



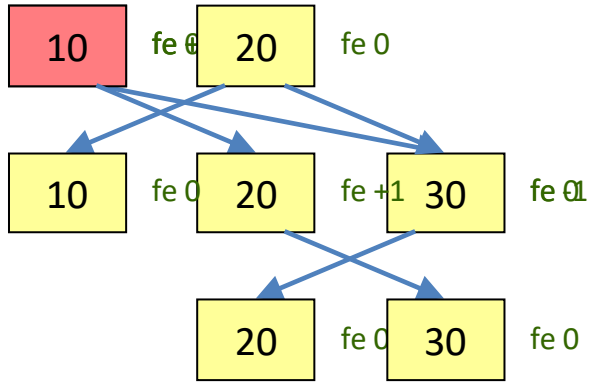
fe(30) = 2 y fe(30.izq) = 1, entonces caso LL

Implementación

Árbol AVL

```
Nodo* reequilibrar(Nodo* n) {
   int fe = factorEquilibrio(n);
   if (fe < -1) {
      if (factorEquilibrio(n->izq) <= 0)
         return rotarDerecha(n);   // LL
      n->izq = rotarIzquierda(n->izq);
      return rotarDerecha(n);      // LR
   }
   if (fe > 1) {
      if (factorEquilibrio(n->der) >= 0)
         return rotarIzquierda(n); // RR
      n->der = rotarDerecha(n->der);
      return rotarIzquierda(n);    // RL
   }
   return n;
}
```

caso RL: 10 30 20



Ejercicio 4: la rotación doble que falta

Tu tarea: en 06_arbol_avl.cpp el caso RL está completo y sirve de plantilla. Completa el caso **LR**, simétrico.

Antes de escribir código: dibuja a mano el resultado de insertar 30, 10, 20 (en ese orden) sin ninguna rotación.

¿Qué nodo queda desequilibrado, y hacia qué lado pesa?
¿Basta una rotación simple a la derecha de la raíz para arreglarlo?

La raíz (30) queda con fe −2 (pesa la izquierda), pero el 20 cuelga a la **derecha** del 10. Rotar 30 a la derecha subiría al 10 y dejaría al 20 del lado que no toca: primero hay que **enderezar la rama** (rotar el 10 a la izquierda).

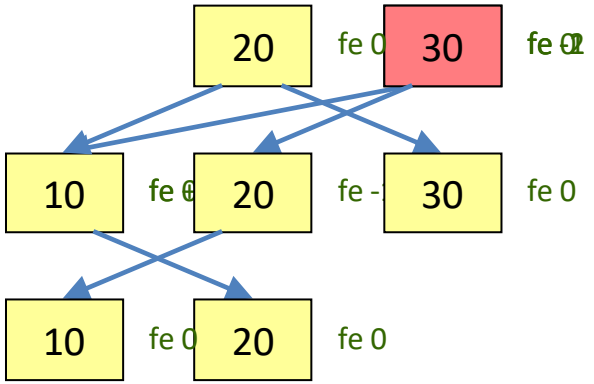
06_arbol_avl.cpp

Implementación

Árbol AVL

```
Nodo* reequilibrar(Nodo* n) {
   int fe = factorEquilibrio(n);
   if (fe < -1) {
      if (factorEquilibrio(n->izq) <= 0)
         return rotarDerecha(n);   // LL
      n->izq = rotarIzquierda(n->izq);
      return rotarDerecha(n);      // LR
   }
   if (fe > 1) {
      if (factorEquilibrio(n->der) >= 0)
         return rotarIzquierda(n); // RR
      n->der = rotarDerecha(n->der);
      return rotarIzquierda(n);    // RL
   }
   return n;
}
```

caso LR: 30 10 20



Implementación

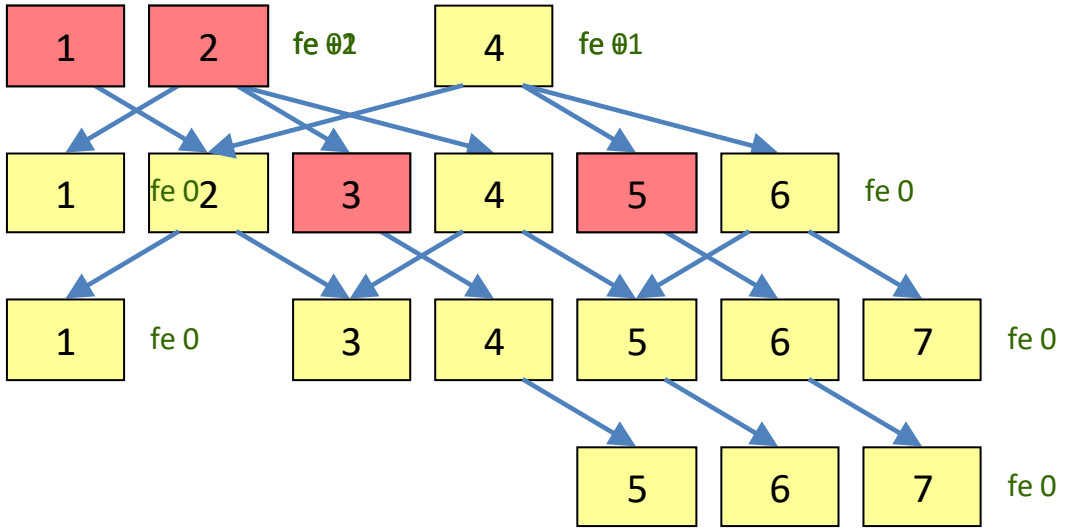
Árbol AVL: el peor caso

```
for (int i = 1; i <= 7; i++)
   avl.insertar(i);    // EL PEOR CASO
                       // del BST simple
```

BST simple: una cadena de altura 6.
AVL: altura 2, el árbol perfecto.

Cada rotación es O(1) y como mucho hay una por inserción.

insertar 1..7 en orden



altura mínima posible

El aviso del fichero 03, arreglado

n (insertados en orden)	altura AVL	cota teórica $1{,}44 \cdot \log_2 n$
100	6	9,6
1000	9	14,4
10000	13	19,1
100000	16	23,9

El BST simple, con los mismos datos, tenía altura **99 999**.

La cota de $1{,}44 \log_2 n$ es la que demostraron Adelson-Velskii y Landis: **nunca se supera**, insertes en el orden que insertes.

06_arbol_avl.cpp

El B-tree

Cuando cada nivel es un acceso a disco

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Cuando el acceso es caro

Todo lo anterior asume que **comparar** dos claves es lo caro: memoria RAM, nanosegundos.

Si el árbol vive en **disco** (un índice de base de datos, un sistema de ficheros),
cada nivel que se baja es **un acceso a disco**: órdenes de magnitud más lento.

Lo que hay que minimizar ya no son las comparaciones: es el número de **nodos visitados**, es decir, la **altura**.

La solución (Bayer y McCreight, 1972): **romper la aridad 2**.
Cada nodo guarda **muchas** claves ordenadas y tiene **muchos** hijos.
Con grado mínimo t = 100, un millón de claves cabe en 2 ó 3 niveles.

La estructura

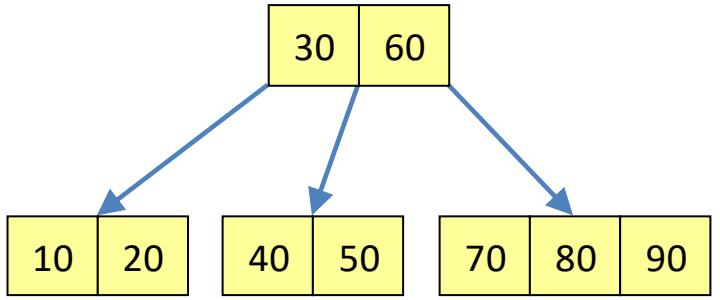
Con un **grado mínimo t**:

Cada nodo (salvo la raíz) tiene entre **t – 1** y **2t – 1** claves, ordenadas.

Un nodo con k claves tiene, si no es hoja, **exactamente k + 1 hijos**.

Las claves **separan** los hijos: lo que cuelga entre c[i−1] y c[i] está entre ellas.

Todas las hojas están al mismo nivel.



ejemplo con t = 2 (de 1 a 3 claves por nodo)

07_arbol_b.cpp

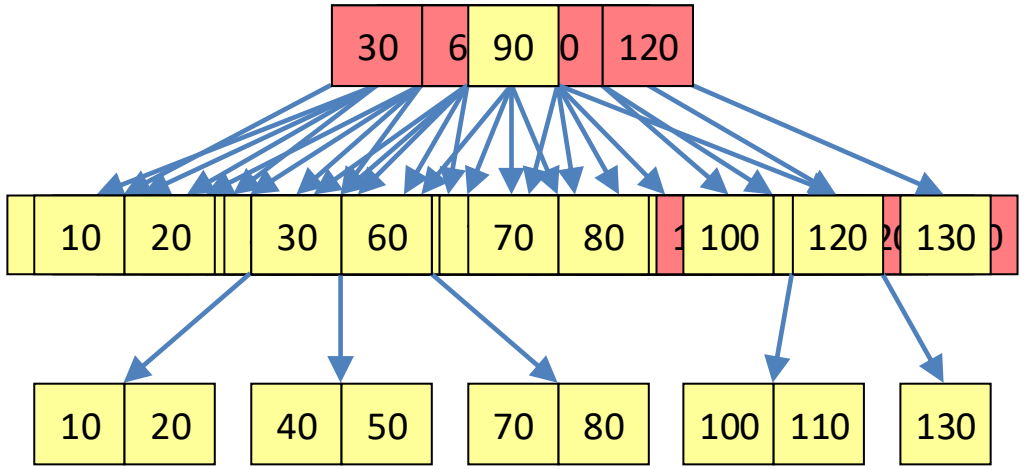
Implementación

B-tree, t = 2 (de 1 a 3 claves)

```
void insertar(int v) {
   // 1. bajar hasta la HOJA que toca
   // 2. meter v ordenado en ella
   // 3. al volver: si un nodo llega a
   //    2t claves, se PARTE y su clave
   //    del medio sube al padre
   // 4. si se parte la raíz, nace una
   //    raíz nueva: el árbol crece
   //    POR ARRIBA
}
```

Por eso todas las hojas están siempre al mismo nivel.

insertar 10, 20, ..., 130



la raíz se 2t+1 (puede

Ejercicio 5: el split

Un nodo nunca puede superar 2t − 1 claves. Si al insertar llega a **2t**, se **parte** en dos: la clave del medio sube al padre como nueva separadora.

Tu tarea: completar dividirSiHaceFalta() en 07_arbol_b.cpp.

Antes de escribir código: un nodo con 2t claves, [c0 … c(2t−1)].

¿Qué índice es "el del medio"?

¿Cuántas claves quedan a su izquierda y cuántas a su derecha?

¿Encaja eso con los límites t − 1 y 2t − 1 del invariante?

07_arbol_b.cpp

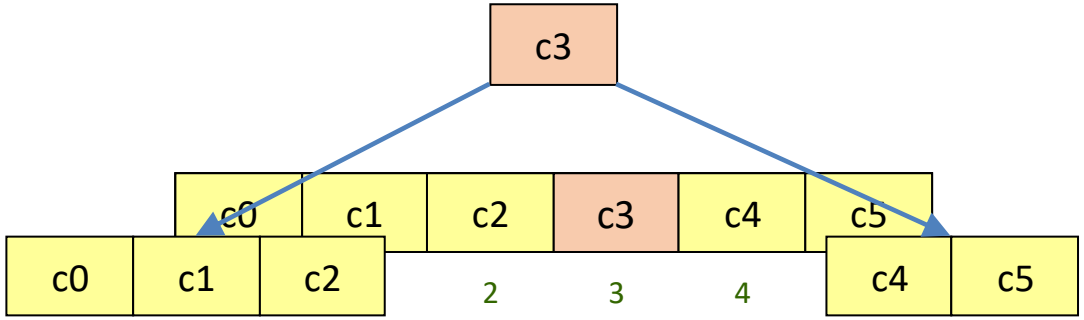
Implementación

B-tree, en general (t = 3)

```
bool dividirSiHaceFalta(Nodo* n,
        int& sube, Nodo*& hermano) {
    if (n->claves.size() < 2 * t_)
        return false;
    // TODO (ejercicio 5):
    //  - la clave de índice t_ sube
    //  - el resto, en dos nodos
    //  - y sus hijos, si no es hoja
    return true;
}
```

Si n no es hoja, los hijos se parten igual: los t+1 primeros a la izquierda (k claves -> k + 1 hijos).

dividirSiHaceFalta()



el de 2 claves [índice 0..1] hasta el de 3 claves [índice 0..2]

Y 2 claves [índice 3..4] hasta el de 2 claves [índice 0..1]

¿Por qué compensa un grado grande?

n	niveles B (t = 100)	altura AVL (≈ 1,44 $\log_2 n$)	insertar B (ms)
1000	1	14,4	0,1
100 000	2	23,9	19,8
1 000 000	2	28,7	114,8
5 000 000	3	32,0	940,1

Con cinco millones de claves, **tres niveles** bastan. Un AVL necesitaría más de treinta.

Si cada nivel es un acceso a disco, es la diferencia entre una consulta instantánea y una perceptible. Por eso un B-tree (o su variante B+) es lo que hay debajo de los **índices de las bases de datos** y de muchos **sistemas de ficheros**.

07_arbol_b.cpp

Los adaptadores de la STL

Los TADs del tema, hechos tipo

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Los TADs del tema, en la STL

Igual que en el tema 3 std::stack y std::queue eran la Pila y la Cola:

std::priority_queue es el montículo del fichero 04, sobre un std::vector.
push = insertar (sube), pop = extraerMaximo (baja), top = maximo

std::set y **std::map** son un árbol de búsqueda **equilibrado**
un **árbol rojo-negro**, no un AVL, convertido en tipo.
Recorrerlos con un iterador da las claves **en orden**: es el inorden.

B-tree: no está en la STL. Solo compensa cuando cada nodo es un acceso a disco; es cosa de bases de datos, no de una biblioteca en RAM.

08_adaptadores_stl.cpp

set y map: el árbol de búsqueda que NO degenera

Insertando **en orden**, el peor caso del BST simple del fichero 03:

n	**set::insert (ms)**	**altura simulada**
1000	0,110	~19 (cota rojo-negro: $2 \cdot \log_2 n$)
80 000	7,645	~32 (cota rojo-negro: $2 \cdot \log_2 n$)

Nunca se dispara. En el BST simple, los mismos datos daban una **cadena de altura n**.

El rojo-negro admite una cota algo más floja que el AVL ($2 \log_2 n$ frente a $1{,}44 \log_2 n$) a cambio de **rotaciones acotadas** al insertar y al borrar: la STL no sabe si vas a buscar mucho o a modificar mucho, y elige el peor caso de modificación más barato.

08_adaptadores_stl.cpp

La tabla final del tema

TAD	En la STL	Por debajo	Por qué
Cola de prioridad	priority_queue	Montículo sobre vector	Acceso i -> 2i+1 y memoria contigua
Conjunto ordenado	set	Árbol rojo-negro	Orden barato + O(log n) garantizado
Diccionario	map	Rojo-negro de pares	Igual que set, con clave y valor
B-tree	(no está)	—	Solo compensa en disco: bases de datos

Fuera de clase: árboles en C++ de verdad

En clase, new / delete y punteros crudos a propósito: así se ve el mecanismo.

https://es.wikipedia.org/wiki/%C3%81rbol_rojo-negro

1. **La STL no trae un tipo "árbol"**: trae TADs hechos con árboles.
   set / map son rojinegros: nodo con padre, izq, der y color, y un centinela.
   Las posiciones son **iteradores**: ++it avanza en inorden subiendo por el padre.

2. **Árbol general con propiedad automática** (la representación A, sin liberar()):
   std::vector<std::unique_ptr<Nodo>> hijos; Nodo* padre;
   Así son el árbol sintáctico de un compilador, el DOM o un grafo de escena de videojuego

3. **Árbol en un array, con índices**: std::vector<Nodo>, hijos como int.
   Sin un new por nodo y amigo de la caché: la idea del montículo, generalizada.

4. **Bibliotecas**: Boost.PropertyTree (JSON / XML), Boost.Graph (si deja de ser árbol).

08_adaptadores_stl.cpp

Cómo elegir, en orden

1. ¿Solo necesito **el extremo** (el mayor, el más urgente)? -> **priority_queue**

2. ¿Necesito **buscar por valor** y **recorrer en orden**? -> **set / map**

3. ¿Son **millones de claves en disco**? -> un **B-tree** (el motor de la base de datos)

4. ¿Es una **jerarquía** sin orden (ficheros, expresiones, el DOM)? -> un árbol a medida,
   y sus **recorridos**

Y siempre la pregunta del tema 3: ¿cuál es la estructura **más restrictiva** que todavía resuelve mi problema?

Resumen

El **TAD Árbol** es la jerarquía sin decir cómo se guarda; en el binario, el vacío es parte de la definición, y en el BST el TAD pasa a ser un Conjunto.

Un **árbol n-ario** siempre cabe en uno binario (primer hijo / hermano derecho).

Un **árbol binario** no tiene orden: los recorridos son propiedades de la **forma**.

El **BST** busca en O(log n)… si está equilibrado. Insertar en orden lo degenera.

El **montículo** garantiza O(log n) **por la forma**, renunciando a buscar por valor. Construirlo cuesta O(n).

El **AVL** repara el BST con rotaciones: nunca más de 1,44 $\log_2$ n de altura.

El **B-tree** minimiza niveles, no comparaciones: es el índice de una base de datos.

La STL: priority_queue = montículo; set / map = **rojo-negro**.